Hitesh Rathore (HR) Notes

LINKEDIN LINK:
https://www.linkedin.com/in/hitesh-rathore-835279305/

CHAPTER 1

A WORLD BEFORE THE WEB

HOW COMPUTERS WORKED **BEFORE** INTERNET, WEBSITES & JAVASCRIPT EXISTED

1 OPENING SCENE

YEAR: 1985

Hitesh wants to use Microsoft Word.

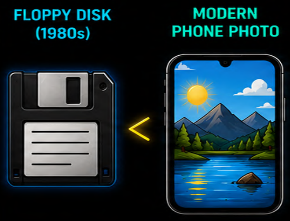
SOFTWARE SHOP

- ❌ No App Store
- +
- ❌ No Download
- +
- ❌ No Internet

Software Shop → Floppy Disk → Carry Home → Install in PC

Software had to physically travel before the Web existed.

2 FLOPPY DISK ERA



FLOPPY DISK (1980s)	MODERN PHONE PHOTO
Capacity: 1.44 MB	Size: ≈ 5 MB

One Photo Today
Can Be Larger Than
An Entire Floppy Disk!

3 INSTALLING SOFTWARE

Hitesh buys Microsoft Word.

Insert Disk
↓
Copy Files
↓
Hard Drive
↓
Install Program
↓
Open Word

Everything stays inside your machine.

4 WHAT DOES "LOCAL" MEAN?

Everything happens on **YOUR** computer.

YOUR COMPUTER

- CPU
- RAM
- Hard Drive
- Microsoft Word

No Internet | No Server | No Cloud

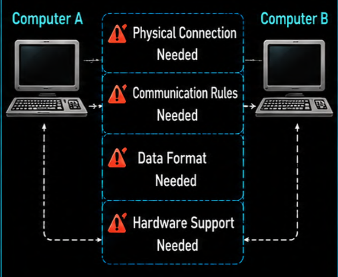
Story: Hitesh types "Hello World" in Word.

Type → Word Processes → Save → Display

Everything happens inside the same box.

5 WHY WAS EVERYTHING LOCAL?

Connecting computers was difficult.



Computer A ↔ Computer B:

- ⚠ Physical Connection Needed
- ⚠ Communication Rules Needed
- ⚠ Data Format Needed
- ⚠ Hardware Support Needed

So most software was designed for:

One User · One Computer · One Program

6 SOFTWARE UPDATE PROBLEM

Before Internet:

Bug Found
↓
Create New Disk
↓
Ship Worldwide
↓
Customer Reinstalls

Result:

- Slow
- Expensive
- Painful

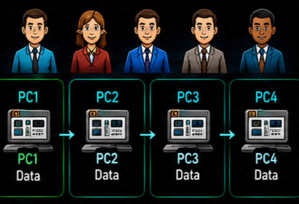
7 SHARING FILES WAS HARD

THEN (Old Days)	NOW (Modern Days)
Save File ↓	File ↓
Copy To Disk ↓	Send ↓
Give Disk ↓	Received Instantly
Other Opens	

Physical transfer vs Instant sharing

8 COMPANY PROBLEM

Company with 1000 Employees

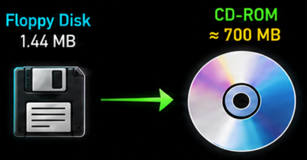


Why doesn't everyone see the same information?

Problems:

- ❌ No Synchronization
- ❌ No Central Database

9 CD-ROM REVOLUTION



Huge Jump in Storage!

Used for:

Games · Windows · Encyclopedia Software · Microsoft Office

10 CLIENT-ONLY WORLD

Everything existed in one place.

USER COMPUTER

- Program
- Data
- Processing
- User

Also called:

Standalone Computing

Local Computing

Analogy: House

Everything is stored inside the house.

11 THE BIG QUESTIONS

QUESTION 1	QUESTION 2	QUESTION 3
What if computers could communicate?	What if information could be shared instantly?	What if software could be delivered through a network?

12 THE FUNDAMENTAL PROBLEM

The pre-Web world had 3 major limitations.

1. **SOFTWARE DISTRIBUTION**
   Need Physical Disks
2. **DATA SHARING**
   Need Physical Transfer
3. **UPDATES**
   Need Reinstallation

CHAPTER 2

THE INTERNET BEGINS TO GROW

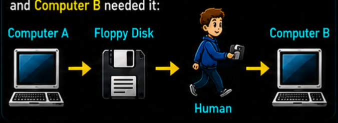
HOW COMPUTER NETWORKS **EVOLVED** & LAID THE FOUNDATION OF **THE INTERNET**

1 IMAGINE A WORLD OF ISOLATED COMPUTERS

In the 1980s, most computers looked like this:

Computer A
Computer B
Computer C
Computer D

If **Computer A** had a file and **Computer B** needed it:



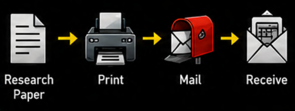
Humans were acting as the "network". This was **slow and inefficient.**

2 THE FUNDAMENTAL NEED

People realized:
Information becomes **more valuable** when it **can be shared.**

Professors
Students
Researchers
Libraries

Without networking:

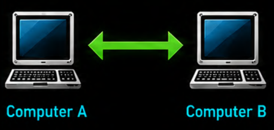


Could take days or weeks.

3 WHAT IS A NETWORK?

At the most basic level:

A network means:
Two or more computers that can exchange data.



That's it.
A network is simply a way for computers to **talk.**

4 WHAT IS DATA?

Computers only understand:

0 and 1

So when a computer sends:

Hello

Internally it sends:

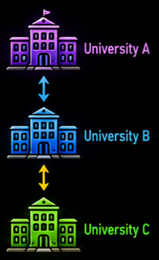
01001000
01100101
01101100
01101100
01101111

Networking is really:
Moving bits from one computer to another computer.

5 EARLY NETWORKS

Universities started connecting computers.

Example:



Researchers could now:

- Share papers
- Share data
- Send messages

without mailing disks.

This was **revolutionary.**

6 WHY GOVERNMENTS BECAME INTERESTED

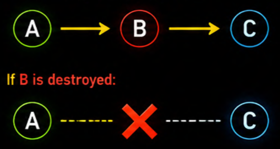
Now think from a military perspective.

Suppose:

Military Base A
Military Base B
Military Base C

What happens if one connection fails?

Traditional systems looked like:



Communication breaks.

7 THE BIG IDEA OF ARPANET

The U.S. government funded a project called:

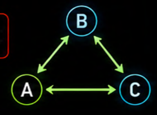
ARPANET

The idea: Create a network that can survive failures.

Instead of:

A → B → C

Use:

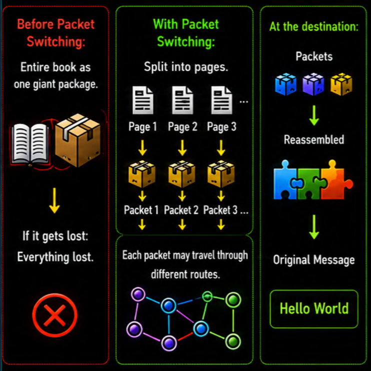


Multiple routes.
If one path fails:
A can still reach **C** through another route.

This concept later became the foundation of the **Internet.**

8 PACKET SWITCHING

One of the greatest inventions in networking.



Before Packet Switching:
Entire book as one giant package.
If it gets lost:
Everything lost.

With Packet Switching:
Split into pages.

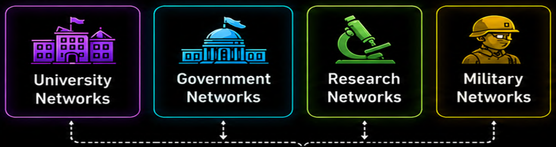
Each packet may travel through different routes.

At the destination:
Packets
Reassembled
Original Message
Hello World

This is how the **Internet** still works today.

9 THE INTERNET IS BORN

Many networks started appearing:



Problem:
They all used different technologies.

How can they communicate?

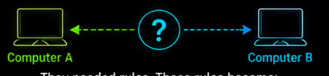
THE BIRTH OF TCP/IP

The technology that allowed different networks to become one giant network:

THE INTERNET

10 TCP/IP CHANGES EVERYTHING

How should Computer A send data to Computer B?



They needed rules. Those rules became:

TCP/IP

the language of the Internet.

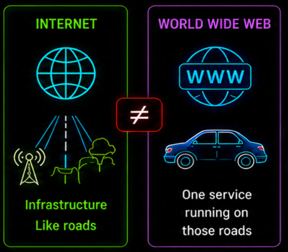
IP's Job (Addressing)
IP gives computers addresses.
192.168.1.10
Like a house address.
Without an address, data has nowhere to go.

TCP's Job (Reliability)
TCP ensures data arrives correctly.
1 – 2 – 3 – 4 – 5 – 6
If Packet 5 is missing, TCP requests Packet 5 again.
This made communication reliable.

★ TCP/IP = Language of the Internet

11 INTERNET ≠ WEB

This is where many beginners get confused.



INTERNET
Infrastructure
Like roads

WORLD WIDE WEB
One service running on those roads

NOT THE SAME THING

★ You can have roads without cars. Similarly, the **Internet existed before websites.**

12 WHAT WAS INTERNET USED FOR BEFORE THE WEB?

Mostly:

- Email
- Send Messages
- File Transfer
- Share Files
- Research
- Exchange Information
- Military Communication
- Secure Communication

Ordinary people rarely used it.

Mostly used by:

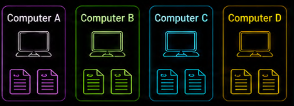
- Scientists
- Universities
- Governments

13 THE PROBLEM TIM BERNERS-LEE SAW

At the research organization:

CERN
Researchers had huge amounts of information.

Problem:
Documents everywhere.

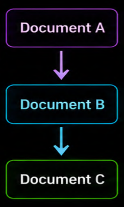


Nobody knew where everything was. Finding information was difficult.

14 TIM BERNERS-LEE'S REVOLUTIONARY THOUGHT

Tim Berners-Lee asked:

What if documents could reference other documents?



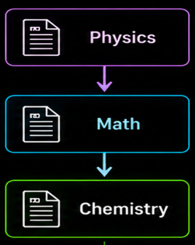
Like a giant connected knowledge system.

You click and move between them.

This sounds normal today.
In 1990, it was a **revolutionary idea.**

15 HYPERLINKS

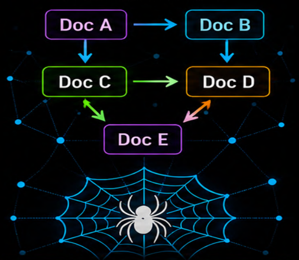
The core idea:



★ Every document could connect to another document.
This created a **giant web.**

16 WHY IT'S CALLED THE WORLD WIDE WEB

Imagine millions of documents connected like this:



It looked like a **Spider Web.**
Therefore:
World Wide Web

17 THREE TECHNOLOGIES CHANGED HISTORY

Tim Berners-Lee created three foundational technologies:

HTML
For creating documents.
- Content
- Paragraphs
- Headings
- Links

URL
For locating documents.
Example: https://example.com
An address for information.

HTTP
For transferring documents.
Browser: Give me this document.
Server: Here it is.

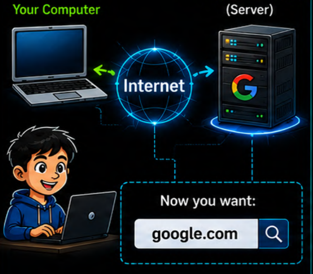
HTTP DETAILING

Understanding the Foundation of the Web

1 IMAGINE TWO COMPUTERS

You have:

Somewhere else:



Now you want:

google.com

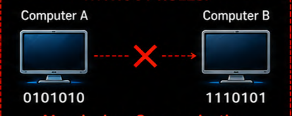
How will your computer ask for it?

2 THE FUNDAMENTAL PROBLEM

Your computer cannot magically know:

- Which file you want
- Which website you want
- What format you want
- Whether you're sending data or receiving data

WITHOUT RULES:



Meaningless Communication

There must be **rules.**
Just like humans need language.

3 WHAT IS A PROTOCOL?

A protocol is simply:
An agreed set of communication rules.

Traffic Rules

Red Light → Stop
Green Light → Go

Everyone follows the same rules.

Human Language

"Hello" has a common meaning.

Computer Protocol

☑ How to start communication?
☑ How to ask for data?
☑ How to send data?
☑ How to know communication ended?

★ Computers also need rules.
Those rules form a **PROTOCOL.**

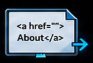
4 MEANING OF HTTP

HTTP stands for:

HyperText Transfer Protocol

HyperText

In the early web, a document could contain links.



One document could jump to another.

This linked text was called HyperText

Not just text. Text connected to other text.

Transfer

Means: Move data from one computer to another computer.

Protocol

The rules.

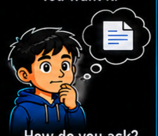
So **HTTP** means:
Rules for transferring hypertext documents across a network.

5 BEFORE HTTP EXISTED

Imagine the year 1990.

A document exists on:
Server Computer

You want it.



How do you ask?

THE PROBLEM:

Different Systems	Different Methods	Different Rules

Tim Berners-Lee wanted:
Any computer can request any document from any server.

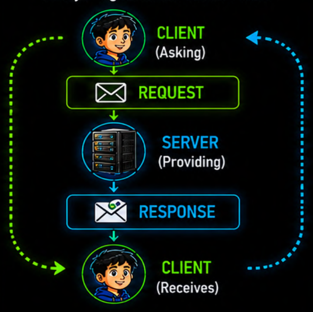
★ **So he designed HTTP.**

6 THE MOST IMPORTANT CONCEPT

HTTP follows:

REQUEST → RESPONSE

Everything revolves around this.



★ Everything in HTTP is Request → Response.

7 WHO IS CLIENT?

Client means:
The computer **asking** for something.

Give me google.com

Chrome asks for google.com

Chrome is the client. ✓

Examples of Clients:

Chrome Firefox Edge Safari Any App

★ Client = The thing that **asks** for data.

8 WHO IS SERVER?

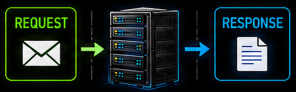
Server means:
The computer **providing** something.

Google Server

Stores:

- Web Pages
- Images
- Videos
- Data

When a request arrives:



Server → Sends Page

★ Server = The thing that **provides** data.

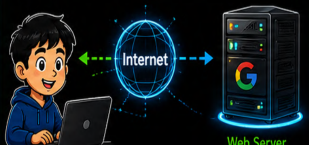
9 WHERE DO WEBSITES LIVE?

Websites live on special computers called Web Servers.

You type:

www.example.com

Behind the scenes:
Your request goes to the Web Server where the website lives.



Web Server
(Stores Website)

★ **Web Server** = A computer that stores and delivers websites.

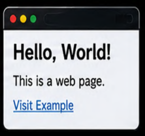
10 WHAT IS A WEB PAGE?

A web page is a document written in **HTML**.
It tells the browser what to show.

HTML (Example)

```
<html>
<head> <title>My Page</title> </head>
<body>
  <h1>Hello, World!</h1>
  <p>This is a web page.</p>
  <a href="https://example.com">
    Visit Example</a>
</body>
</html>
```

Browser reads HTML and shows it as a web page. →



Hello, World!

This is a web page.

Visit Example

★ **HTML** = The language of web pages.

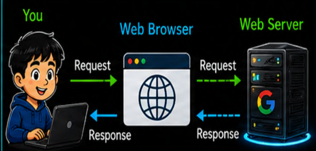
11 HOW DO WE SEE WEBSITES?

We use a **Web Browser** to request and see web pages.

Popular Browsers

Chrome Firefox Edge Safari

and more...



★ **Browser** = The tool that gets web pages from servers.

12 LET'S USE A COOK EXAMPLE

Think of a restaurant:

YOU (CLIENT)		SERVER (RESTAURANT)
You go to a restaurant		Restaurant is ready (server is on)
You choose (request) food		Chef prepares the food (process request)
You receive (response) food		Chef serves the food (send response)
You enjoy the meal		You are satisfied!

★ Same idea: Request → Process → Response
That's how the web works!

9 WHAT ACTUALLY HAPPENS?

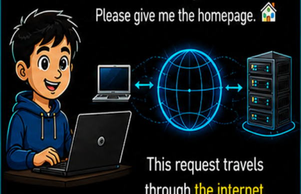
You type:

https://google.com

Chrome creates a request.

Something like: `GET /`

Meaning:
Please give me the homepage.



This request travels through the internet.

10 SERVER RECEIVES REQUEST

Google's server receives:

`GET /`

and thinks:

User wants homepage.

Then it prepares data...

11 RESPONSE

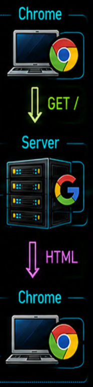
Server sends back:

`HTTP/1.1 200 OK`

plus the HTML.

Example:

```
<html>
  <body>
    Google
  </body>
</html>
```



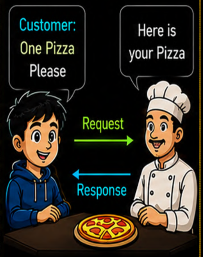
★ This is **HTTP.**

12 WHY REQUEST-RESPONSE IS GENIUS

The server does NOT continuously send data.

Client asks
↓
Server replies
↓
Conversation ends

RESTAURANT ANALOGY

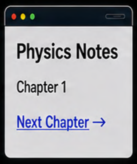


★ HTTP works similarly.

13 WHY HTML WAS ENOUGH INITIALLY

Early websites looked like:

```
<h1>Physics Notes</h1>
<p>Chapter 1</p>
<a href="chapter2.html">
  Next Chapter
</a>
```



Physics Notes

Chapter 1

Next Chapter →

No Animations	No Forms	No Login	No Shopping Carts	No Video Streaming

★ HTTP was originally built to transfer these documents.

14 WHY HTTP IS STATELESS

Request 1:
GET /home
↓
Server Responds
↓
Connection ends.

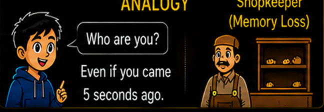
Later...

Request 2:
GET /about
↓
Server Responds
↓
Connection ends.

HTTP assumes:
Every request is independent.

Server forgets previous requests.

ANALOGY


Shopkeeper (Memory Loss)
Who are you?
Even if you came 5 seconds ago.

★ This is called **STATELESSNESS.**

15 THEN HOW DO WEBSITES REMEMBER ME?

Websites like:

Instagram Gmail Amazon

remember who you are.

But HTTP is stateless.
How?

- Cookies
- Sessions
- Tokens

★ These were added later.
Originally HTTP had no memory.

16 A REAL-LIFE WEBSITE LOAD

Suppose: https://example.com contains:

HTML CSS JavaScript Images Fonts

The browser performs many HTTP requests.

Request HTML	Request CSS	Request JS	Request Image	Request Font
Receive HTML	Receive CSS	Receive JS	Receive Image	Receive Font

★ Modern pages may trigger **hundreds** of HTTP requests.

THE MOST IMPORTANT MENTAL MODEL

Browser
↓
“Can I have this resource?”
↓
Server
↓
“Sure, here it is.”
↓
Browser

★ That simple conversation is **HTTP.**

THE MOST IMPORTANT INSIGHT

Before 1990:

Internet

Mostly a communication network for researchers.

After Tim Berners-Lee:

Internet
+
Hyperlinked Documents
=
World Wide Web

For the first time, anyone could navigate information by simply clicking links.

This transformed the Internet from

A NETWORK FOR EXPERTS → **A GLOBAL INFORMATION SYSTEM**

CHAPTER 3

BROWSERS BECOME POPULAR

1 WHY WAS A BROWSER NEEDED?

A new problem appeared.

Suppose a server sends HTML:

```
<h1>Physics</h1>

<a href="math.html">
  Math
</a>
```

SERVER

How would a computer know:

- What is a heading?
- What is a link?
- What should happen when someone clicks?

A software was needed that could understand HTML. → That software became: **Browser**

2 THE FIRST WEB BROWSER

The first web browser was:

WorldWideWeb

Created by:

Tim Berners-Lee

around **1990** while working at **CERN.**

WorldWideWeb Browser

CERN

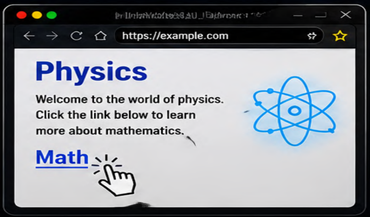
THE WEB: FROM AN IDEA TO A GLOBAL REVOLUTION

1989 Idea of the World Wide Web → **1990** WorldWideWeb Browser Created → **1991** Web Made Public → The Internet Starts Growing

3 WHAT DID THE WORLDWIDEWEB BROWSER DO?

Its job was:

Receive HTML
↓
Interpret HTML
↓
Display Document
↓
Allow Clicking Links



https://example.com

Physics

Welcome to the world of physics. Click the link below to learn more about mathematics.

Math

★ This is still the **fundamental job** of browsers today.

4 WHY DID THE MOSAIC BROWSER BECOME FAMOUS?

The WorldWideWeb browser could only run on a limited number of systems.

At that time, computers used different operating systems.

Examples:

- Unix
- Windows
- Mac

The WorldWideWeb browser was not available everywhere.

Mosaic browser solved this problem.

MOSAIC BROWSER:

- Supported multiple operating systems
- Displayed images better
- Provided an easy-to-use interface

mosaic
The Web Browser

RESULT:

More Users
↓
More Websites
↓
Web Explosion

Then a company called:
Netscape
released:
Netscape Navigator
in **1994.**

★ This changed everything.

People suddenly wanted websites. **Millions** of new users appeared.

CHAPTER 4

WHY JAVASCRIPT WAS CREATED

</>
JS
{ }

THE REAL STORY BEHIND JAVASCRIPT

1 THE WEB BEFORE JAVASCRIPT (1991–1993)

Tim Berners-Lee created:

- HTML
- HTTP
- URLs

WWW

A webpage looked like:

```
<h1>Welcome</h1>
<p>Hello World</p>
<a href="page2.html">
  Next Page</a>
```

BROWSER WORKFLOW:

Request HTML → Receive HTML → Display HTML

That’s it.

AT THAT TIME:

Web = Online Documents

Examples:

- Research Papers
- University Notes
- Encyclopedias
- Digital Libraries

PEOPLE WERE NOT IMAGINING:

Gmail · YouTube · Netflix · Instagram · Google Maps

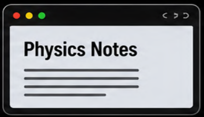
The web was viewed as a collection of documents, not applications.

2 EARLY BROWSERS WERE DOCUMENT READERS

Suppose a page contains:

```
<h1>Physics Notes</h1>
```

- Browser receives HTML.
- Browser displays HTML.



QUESTION:	
Can it calculate?	✕ No
Can it validate forms?	✕ No
Can it react to user actions?	✕ No
Can it remember state?	✕ No
Can it run programs?	✕ No

PDF Reader

PDF → Read
→ Close

SIMILARLY:

→ Display Page

- No logic.
- No behavior.
- No intelligence.

3 MOSAIC BROWSER (1995)

One of the first popular graphical browsers was:

MOSAIC

It proved that:

People actually enjoy browsing websites.

MOSAIC INTRODUCED:

Images on webpages · Hyperlinks · Easier navigation

BEFORE MOSAIC:
Internet was mostly text-based.

→

AFTER MOSAIC:
The web became more user-friendly.

4 NETSCAPE NAVIGATOR ARRIVES (1994)

Some important Mosaic developers left NCSA.

One of them was:

Marc Andreessen

They founded:

Netscape Communications

and created:

Netscape Navigator

COMPANY vs BROWSER

Company	Browser
Google (Company) →	Chrome (Browser)
Microsoft (Company) →	Edge (Browser)
Apple (Company) →	Safari (Browser)
Netscape (Company) →	Netscape Navigator (Browser)

★ Important:
Netscape = Company | Netscape Navigator = Browser

5 WHY NETSCAPE BECAME SO POPULAR

Navigator was much better than Mosaic.

MOSAIC:
Downloading...
Downloading...
Downloading...
↓
Page Appears

→

NAVIGATOR:
Downloading...
↓
Text Appears
↓
Images Load
↓
Page Complete

RESULT:
Internet Usage Exploded
↓
Millions of new users arrived.

At one point Netscape controlled:
75–90%
Browser Market Share

Users could start reading immediately.
This was a huge improvement on slow internet.

Think of it as: Netscape Navigator (1995) ≈ Chrome Today

6 BUSINESSES DISCOVER THE WEB

As users increased, businesses started asking:

- Can we sell products online?
- Can users register?
- Can we collect customer data?
- Can we provide services online?

The web changed from:

Web = Information Platform TO Web = Business Platform

7 A HUGE PROBLEM APPEARS

Early websites worked like this:

Request →
↓
Server →
↓
Response →
↓
Browser →

Everything required talking to the server. Even very small tasks.

Imagine:
2 + 2
Could the browser calculate it?
✕ No
The request had to go to the server.

EXAMPLE:
↓
"Please calculate 2 + 2"
↓
Server
4
↓
Browser

Think about a restaurant.

You ask the chef: "What is 2 + 2?" → Chef replies: "4"

Ridiculous. Yet websites worked in a very similar way.

Every tiny task depended on the server.

8 THE FORM VALIDATION PROBLEM

A website form:
Name
Email
Password
SUBMIT

User enters:
Name = Hitesh
Email = abc
Password = 123

WITHOUT JAVASCRIPT:
Browser
↓
Send Form To Server
↓
Server Validation
↓
Error Response
↓
Page Reload

SERVER CHECKS:
Email = abc
Missing "@"
Invalid.

→

SERVER SENDS:
"Email Invalid"
Browser reloads the page.

9 WHY THIS WAS A SERIOUS PROBLEM

Internet in 1994 was very slow.

TYPICAL SPEEDS (1994)	TODAY'S SPEEDS
14.4 kbps	100 Mbps
28.8 kbps	500 Mbps
56 kbps	1 Gbps

Every page reload felt painful.
Users often waited:

5 Seconds | 10 Seconds | 20 Seconds

just to see: "Email Invalid"

Users hated this experience.

10 THE SERVER SCALABILITY PROBLEM

Imagine:
10 Users
No problem.

→

Now imagine:
1,000,000 Users

Every validation requires:
Network (Request) → Server (Process) → Network (Response)

just to check:
"Does email contain @ ?"

⚠ This was wasteful.

11 FIRST PRINCIPLES THINKING

Where is the form?
→ Browser

Where is the email field?
→ Browser

Where is the keyboard input? →
→ Browser

Yet:
Browser
↓
Server
↓
Browser

Everything already exists inside the browser.

12 THE LATENCY PROBLEM

A new concept became important:

LATENCY

Meaning: Time taken for communication. →

EXAMPLE:
India
↓
America
↓
India

Even today it takes time.
In 1994 it was much slower.

Companies realized: Some logic should run directly inside the browser.

13 THE MISSING PIECE IN WEB ARCHITECTURE

HTML Structure → HTTP Data Transfer → Server Business Logic → Browser Render Content

QUESTION: Who handles behavior inside the browser?

ANSWER: Nobody. → There was a huge gap in the architecture.

14 WHAT COMPANIES WANTED

- ✓ Validate forms instantly
- ✓ React to button clicks
- ✓ Open menus
- ✓ Perform calculations
- ✓ Update page content
- ✓ Avoid unnecessary server requests

EXAMPLES:
Click Button → Something Happens
Hover Menu → Opens Menus
Quantity × Price → Show Total

</> The browsed not do these things.

15 THE FUNDAMENTAL REALIZATION

ORIGINALLY:
Web = Documents
↓
LATER:
Web = Applications

But browsers still only knew how to display documents.

The industry needed:

A PROGRAMMING LANGUAGE INSIDE THE BROWSER

that could:
- Read user input
- React to events
- Manipulate content
- Reduce server requests

16 BRENDAN EICH AND JAVASCRIPT (1995)

Netscape needed a language for the browser.

BRENDAN EICH

1995
Mocha (Initial Name)
↓
LiveScript (Second Name)
↓
JavaScript (Final Name)

He created JavaScript in just 10 days!

17 WHY THE NAME CHANGED

Mocha
↓
LiveScript
↓
JavaScript
↓
JavaScript
JS

Netscape wanted to ride the popularity of "Java".

Important:
≠
Java ≠ JavaScript

18 THE REVOLUTION

BEFORE JAVASCRIPT:	AFTER JAVASCRIPT:
Browser	Browser
Display Documents	Run Programs

★ The browser went from a passive viewer to an active engine.

THE JOURNEY FROM DOCUMENTS TO APPLICATIONS

1990 — World Wide Web Invented
→ 1991–1993 — Web = Documents (HTML, HTTP, URLs)
→ 1993 — Mosaic Browser People started loving the web
→ 1994 — Netscape Navigator A huge leap in user experience
→ Businesses Want Interactivity — Web becomes a Business Platform
→ 1995 — JS — Mocha Created (Brendan Eich)
→ LiveScript — LS — Name Changed to LiveScript
→ JavaScript — JS — Final Name JavaScript
→ Browser Becomes Programmable — Web = Applications The real revolution

CHAPTER 5

WHAT JAVASCRIPT BROUGHT TO THE BROWSER

WHAT JAVASCRIPT 1.0 DID NOT HAVE
- ✕ React
- ✕ Node.js
- ✕ Promises
- ✕ Async/Await
- ✕ TypeScript
- ✕ ES6 Classes

BUT IT INTRODUCED THE MOST IMPORTANT BUILDING BLOCKS OF PROGRAMMING!

1 VARIABLES

EXAMPLE:

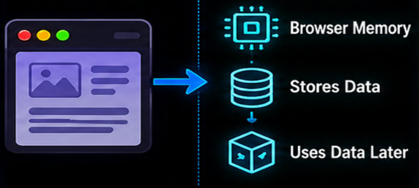
```
var name = "Hitesh";
```

BEFORE JAVASCRIPT

Pages displayed fixed content.

AFTER JAVASCRIPT

The browser could store information.



Allowed pages to remember values while running.

2 CONDITIONS

EXAMPLE:

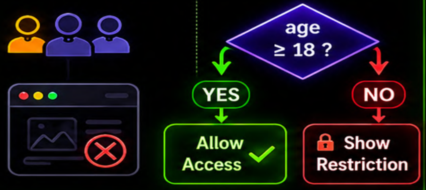
```
if(age >= 18){
    ...
}
```

BEFORE JAVASCRIPT

Every user saw the same behavior.

AFTER JAVASCRIPT

The browser could make decisions.



Smart decisions based on conditions!

3 LOOPS

EXAMPLE:

```
for(var i = 0; i < 10; i++){
    ...
}
```

BEFORE JAVASCRIPT

Perform one action manually.

AFTER JAVASCRIPT

Repeat actions automatically.

Generate lists
Process data
Repeat calculations
Update multiple elements

Repeat anything. Save time. Automate work!

4 FUNCTIONS

EXAMPLE:

```
function greet(){
  alert("Hello");
}
```

FUNCTIONS ALLOW DEVELOPERS TO:

- Reuse logic
- Organize code
- Avoid repetition

BEFORE

Write the same logic repeatedly. → ✗

AFTER

Write once. Use many times. → ✓

Write Once, Use Everywhere!

5 CALCULATIONS

EXAMPLES:

```
2 + 2
10 * 50
100 / 4
```

Now calculations could happen directly inside the browser.

- → User Input
- → JavaScript Runs
- → Result Appears

No server required!

THE FIRST REAL SUPERPOWER

Calculations alone were not revolutionary.

What calculations enabled was revolutionary.

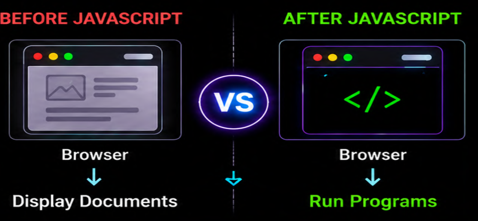
CALCULATIONS LEAD TO:

Validation	Dynamic Content
Comparisons	Games
Decisions	Animations

EXAMPLE IN ACTION:

```
if(email === "")
if(password.length < 8)
price * quantity
score += 10
position = position + 1
```

BEFORE vs AFTER JAVASCRIPT



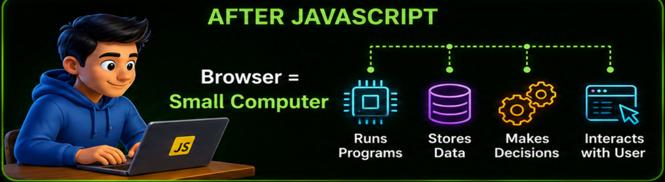
BEFORE JAVASCRIPT	AFTER JAVASCRIPT
Browser ↓ Display Documents	Browser ↓ Run Programs

THE FUNDAMENTAL SHIFT

BEFORE JAVASCRIPT

Browser =
Viewer

AFTER JAVASCRIPT



The browser stopped being a passive document reader.
It became an environment capable of executing programs directly on the user's machine.

ONE-LINE SUMMARY

JavaScript transformed browsers from document viewers into programmable platforms capable of storing data, making decisions, performing calculations, reacting to user actions, and running applications locally.